

# VeroFlow — Full Detailed Flow: What Actually Happens (Code-Level Trace)

This walks through **exactly** what runs, in order, when you click "Run Test" — with the real function names and files involved at each step.

## THE BIG PICTURE (Who Does What)

| Component           | Role                                                             | Has a GUI?                        |
|---------------------|------------------------------------------------------------------|-----------------------------------|
| Emulator            | The virtual phone being tested                                   | ✔ Yes — you see it on screen      |
| Appium              | Background relay — receives commands, sends them to the emulator | ✗ No — just a terminal process    |
| Ollama + Qwen2.5-VL | The "brain" — looks at screenshots, writes test code             | ✗ No — just answers API calls     |
| ChromaDB            | Memory — remembers screens it has labeled before                 | ✗ No — just a local database file |
| FastAPI backend     | The coordinator — calls everything above in the right order      | ✗ No — just an API server         |
| Next.js dashboard   | What YOU actually look at and click                              | ✔ Yes — this is your UI           |

**Key insight:** Only 2 things in this whole system have a visible window — the emulator and your browser dashboard. Everything else (Appium, Ollama, ChromaDB, FastAPI) runs invisibly in terminals, just passing data back and forth.

# STEP-BY-STEP TRACE OF ONE TEST RUN

## Step 0 — You Click "Run Test" in the Browser

In `veroflow-dashboard/app/page.js`, this function fires:

```

[
javascript
const handleRunTest = async () => {
...
const res = await fetch(`${API_URL}/run-test`, {
  method: 'POST',
  body: formData, // contains apk_path + instruction
});
const data = await res.json();
setResult(data);
}

```

This sends an HTTP request to your FastAPI backend at `http://127.0.0.1:8000/run-test`. Nothing has happened on the phone yet — this is just the browser talking to your Python server.

## Step 1 — FastAPI Receives the Request

In `main.py`:

```

[
python
@app.post("/run-test")
async def run_test(apk_path: str = Form(...), instruction: str = Form(...)):
    result = run_full_test(apk_path, instruction)
    return result

```

FastAPI just unpacks the two values (`apk_path`, `instruction`) and hands them straight to `run_full_test()` — the real work happens in `test_executor.py`.

## Step 2 — Appium Connects to the Emulator

Inside `run_full_test()` in `test_executor.py`:

```
python
options = UiAutomator2Options()
options.platform_name = "Android"
options.device_name = "emulator-5554"
options.app = apk_path
options.automation_name = "UiAutomator2"

driver = webdriver.Remote("http://127.0.0.1:4723", options=options)
```

**This is the moment Appium actually gets used.** This code sends a request to `http://127.0.0.1:4723` — that's your Appium server (the one with no GUI, running in Terminal 2). Appium receives this, talks to `adb` behind the scenes, and **installs your APK on the emulator**. On screen, you'll actually see the app open on the virtual phone at this exact moment.

### Step 3 — Phase 1: Understanding Your Instruction (Ollama, Text Only)

```
python
parsed_actions = parse_instruction(instruction)
```

This calls `pipeline/phase1_nlp_parser.py`, which sends your sentence to Ollama:

```
python
response = ollama.chat(
    model='llama3.2',
    messages=[{'role': 'user', 'content': prompt}]
)
```

**What's happening:** Your typed sentence (`"tap the login button"`) goes to Ollama running locally on your PC. Ollama's text model reads it and replies with structured JSON:

```
json
[{"action": "tap", "target": "login button"}]
```

No screenshot involved yet — this step is pure language understanding.

## Step 4 — Screenshot BEFORE the Action

```
python
before_path = f"storage/screenshots/before_{timestamp}.png"
driver.get_screenshot_as_file(before_path)
```

Back to Appium — this tells the emulator "give me a picture of what you're showing right now," and saves it as a PNG file on your hard drive.

## Step 5 — Phase 4: Turning the Screenshot Into Numbers (CLIP)

```
python
vector = get_image_vector(before_path)
```

This calls `pipeline/phase4_clip_encoder.py`, which loads the screenshot and runs it through the CLIP model:

```
python
image = Image.open(screenshot_path).convert("RGB")
image_input = preprocess(image).unsqueeze(0)
with torch.no_grad():
    image_features = model.encode_image(image_input)
```

**What's happening:** The literal pixels of your screenshot get converted into a list of 512 numbers. This has nothing to do with Ollama or AI text generation — CLIP is a completely separate, smaller AI model whose only job is "turn images into comparable numbers."

## Step 6 — Phase 6: Checking Memory (ChromaDB)

```
python
existing_screen = find_similar_screen(vector)
```

This calls `pipeline/phase6_chromadb_rag.py`:

```
python
results = collection.query(
    query_embeddings=[vector],
    n_results=1
)
distance = results['distances'][0][0]
similarity = 1 - distance
```

**What's happening:** Your screenshot's 512 numbers get compared against every screenshot's numbers stored in ChromaDB from previous test runs. If something is 85%+ similar, ChromaDB says "yes, I've seen this before" and hands back the label that was stored for it last time.

**This is a pure math/database lookup — no AI text generation happens here at all.**

## Step 7 — Phase 3: Labeling the Screen (Only If New) — Ollama, Vision This Time

```
python
if existing_screen:
    screen_label = existing_screen["label"]
    result["screen_source"] = "Reused from memory"
else:
    screen_label = label_screen(before_path) # ← only runs if screen is NEW
    store_screen(vector, screen_label, before_path)
```

If Step 6 found nothing similar, THIS runs — `pipeline/phase3_screen_labeler.py`:

```
python
```

```

response = ollama.chat(
    model='qwen2.5vl:3b',
    messages=[{
        'role': 'user',
        'content': prompt,
        'images': [image_bytes]    # ← the actual screenshot bytes
    }]
)

```

**What's happening:** THIS time, Ollama's vision model (not the text-only one from Step 3) actually looks at the screenshot pixels and describes what it sees — "Login Screen, has email field, password field, login button."

**Important distinction:** Step 3 (NLP parsing) and Step 7 (screen labeling) both use Ollama, but they're two different models doing two different jobs — one reads your sentence, the other reads the image.

## Step 8 — Generate the Actual Test Code (Ollama, Vision Again)

```

python
generated_code = generate_appium_code(parsed_actions, before_path, screen_label)

```

Inside `test_generator.py`, this sends **everything gathered so far** to Ollama in one final call:

- The parsed actions from Step 3 (`[{"action": "tap", "target": "login button"}]`)
- The screen label from Step 6/7 (`"Login Screen"`)
- The actual screenshot image bytes

```

python
response = ollama.chat(
    model='qwen2.5vl:3b',
    messages=[{
        'role': 'user',
        'content': prompt,
        'images': [image_bytes]
    }]
)

```

What comes back is real Python code as plain text, e.g.:

```
python
driver.find_element(AppiumBy.XPATH, "//android.widget.Button[@text='Login']").click()
```

At this point, **nothing has happened on the phone yet** — this is still just text sitting in a Python variable.

## Step 9 — Appium Actually Executes That Code

```
python
exec(generated_code, {"driver": driver, "AppiumBy": AppiumBy})
```

**This is the second and final moment Appium does real work.** Python's `exec()` takes that text string from Step 8 and actually *runs* it as code. Since `driver` (your Appium connection) is passed in, the generated line `driver.find_element(...).click()` genuinely executes — Appium sends that command to the emulator, and **you'll see the tap happen live on the emulator screen.**

## Step 10 — Screenshot AFTER, Package Results

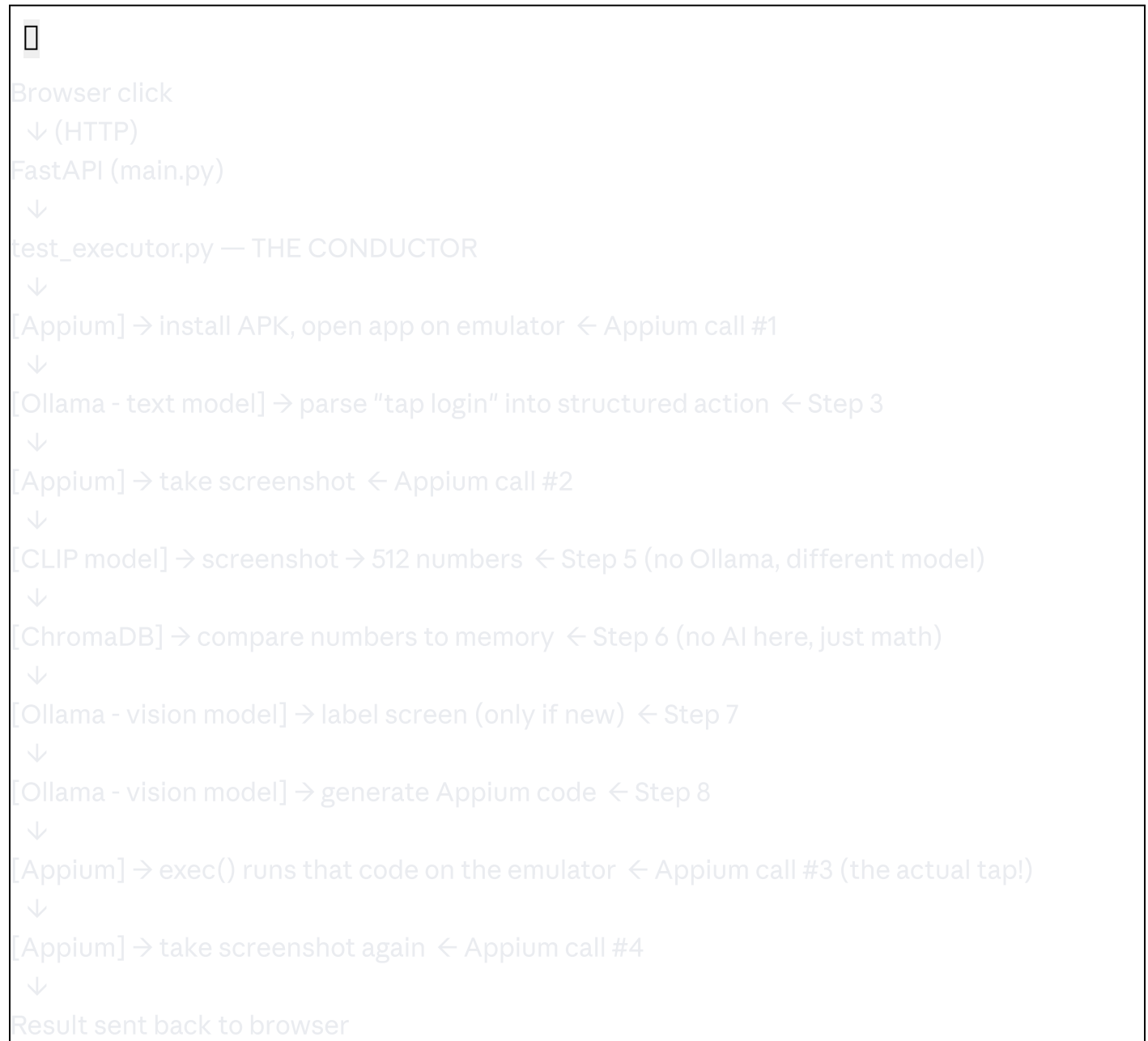
```
python
after_path = f"storage/screenshots/after_{timestamp}.png"
driver.get_screenshot_as_file(after_path)
result["status"] = "passed"
```

Another screenshot (Appium again), then everything collected across all 10 steps gets bundled into one `result` dictionary and returned back up through FastAPI to your browser.

## Step 11 — Dashboard Displays It

Back in `page.js`, `setResult(data)` updates the screen with everything: status, parsed actions, screen label + whether it was reused, the generated code, and both screenshots.

# THE FULL CHAIN, VISUALIZED



**Appium gets called 4 separate times in one test** — install, screenshot, execute, screenshot.  
**Ollama gets called 3 separate times** — parse text, label image (conditionally), generate code.  
**CLIP and ChromaDB get called once each**, and involve no AI text generation at all, just number-crunching and database lookup.

## WHY THIS MATTERS FOR EXPLAINING TO YOUR PROFESSOR

If asked "where does the AI actually make decisions?" — the honest answer is **three distinct points**: understanding the instruction (Step 3), describing what's on screen (Step 7), and writing the automation code (Step 8). Everything else — Appium's four calls, CLIP's vectorization, ChromaDB's lookup — is either mechanical execution or pure math, not "AI deciding things." That's a precise, defensible answer if someone probes on how much of this is "really AI" versus plumbing.